

基于本地大模型与 RAG 技术 的网站智能客服系统

“双碳”主题企业实战项目课答辩汇报

天津大学国际工程师学院

2026 年 6 月 12 日



OUTLINE

1

PROJECT INTRODUCTION

2

PROJECT PROCESS

3

PROJECT RESULT

1 PROJECT INTRODUCTION

项目背景：企业技术资料需要可解释问答

资料分散：NXP AIoT Cloud / Edge AI 资料覆盖平台、工具链、模型和应用范例

提问方式自然化：老师或用户不会严格按照固定FAQ 关键词提问

普通大模型缺少项目内资料约束，容易给出无法追溯的泛化回答

本地 RAG 把答案绑定到知识库片段，并展示标题、来源和相似度

双碳 / 边缘 AI 场景强调本地化、低依赖和可部署的智能服务

资料分散

NXP AIoT Cloud / Edge AI
资料覆盖平台、工具链、模型和应用范例

提问方式自然化

老师或用户不会严格按照固定FAQ 关键词提问

普通大模型缺少项目内

本地 RAG 把答案

技术落点：用本地知识库约束回答范围，答案必须能回到资料标题、来源和检索片段。

1 PROJECT INTRODUCTION

需求与目标

网页端提问、示例问题、Top-K 控制和检索来源展示
健康检查、统计、索引重建、聊天问答等后端接口完整可用

回答同时返回内容、来源、相似度、流程状态和耗时指标

默认本地 Ollama，不依赖云端大模型 API，适合课堂离线演示

通过 AI 创作平台辅助搭建 Agent / RAG 工作流，核心数据与索引落在本地

模型或向量库异常时可降级，不让演示流程中断

网页端提问、示例问题

健康检查、统计、索引

回答同时返回内容、来源

默认本地 Ollama

验收口径：能问、能检索、能生成、能追溯、能降级、能重新构建索引。

2 PROJECT PROCESS

总体架构

前端交互层：HTML / CSS / JavaScript 三栏页面，负责提问、状态和来源展示

后端服务层：FastAPI 接口、Pydantic schema、流程状态和异常处理

RAG 检索层：文档加载、chunk、embedding、ChromaDB、混合召回和来源聚合

本地模型层：Ollama qwen2.5:7b 生成回答，nomic-embed-text 生成向量

数据层：Markdown 知识库和本地向量索引均保存在项目目录内

前端交互层

HTML / CSS / JS

后端服务层

FastAPI API

RAG 检索层

Embedding + Chroma

本地模型层

Ollama LLM

请求流 | 浏览器 →
FastAPI → RAG →
Ollama

数据流 | Markdown → Chunks
→ Embedding → Vector DB

代码入口：app/main.py；核心模块：retriever.py / prompt_builder.py / vector_store.py / ollama_client.py。

2 PROJECT PROCESS

本地知识库构建

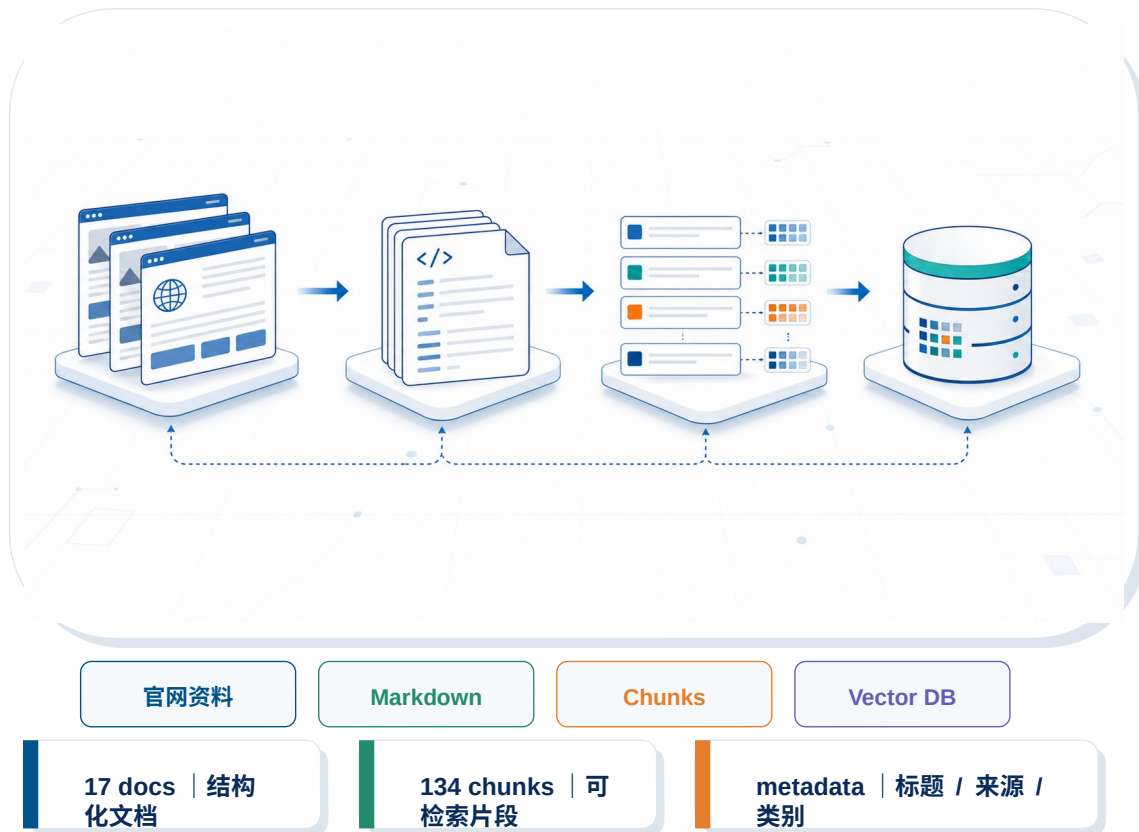
采集 NXP AIoT Cloud、Cloud Lab、AI Hub、Model Zoo 等资料

整理为 17 篇 Markdown 文档，包含标题、分类、来源、摘要和正文

按 chunk 切分为 134 个片段，并保存 metadata 便于追溯来源

向量化后写入 ChromaDB；不可用时自动回退 JSON + numpy 检索

覆盖 LLM/VLM Edge Studio、Qwen/VSS、YOLOv8n、i.MX/MCU 等主题



入库参数：17 docs / 134 chunks；CHUNK_SIZE=500，CHUNK_OVERLAP=80，TOP_K 默认 3。

2 PROJECT PROCESS

RAG 流程与混合检索

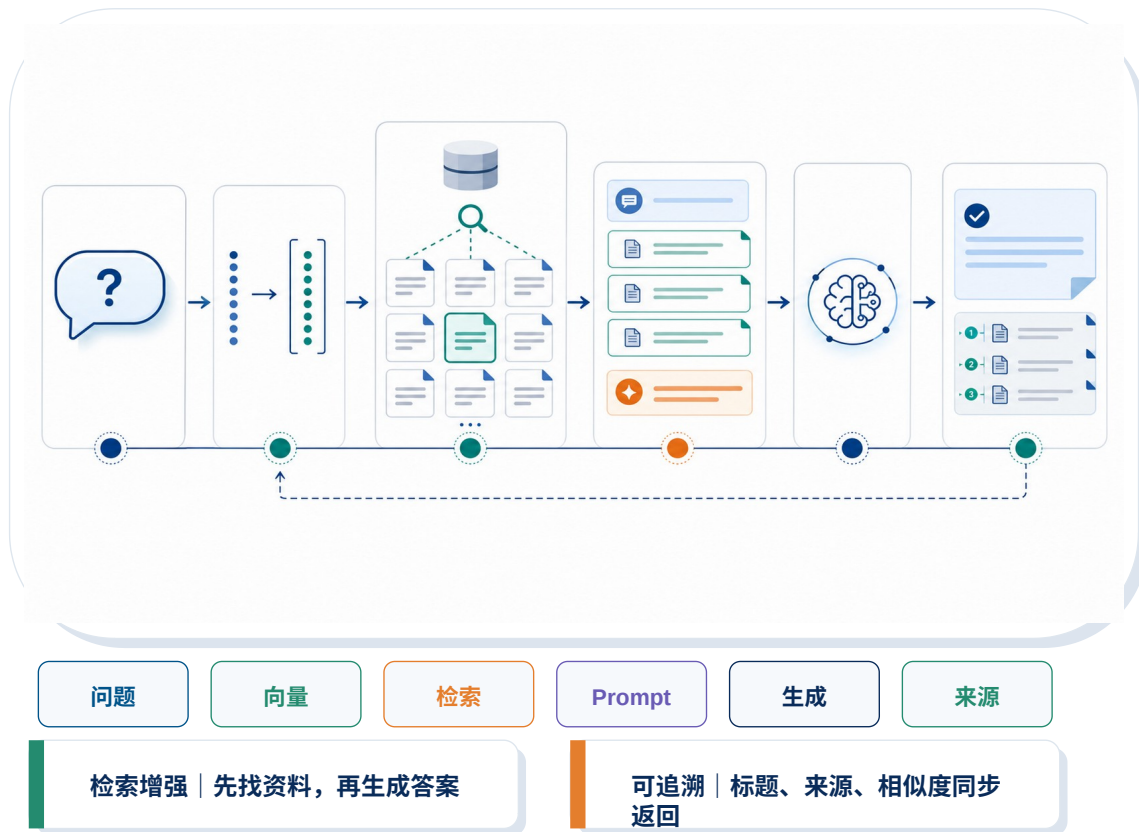
接收问题 → 向量化 → 检索 → Prompt → 本地模型生成 → 返回答案

向量检索负责语义相似度，适合长问题和概念解释类问题

关键词 / 标题匹配补强专有名词、短中文问题和资料数量类问题

按文档聚合来源，避免同一文档多个 chunk 重复刷屏

无明确依据时拒答并清空无关来源，减少“看起来有引用”的误导



召回策略：向量相似度 + 关键词 / 标题加权；低相关结果不进入最终来源。

2 PROJECT PROCESS

本地模型与降级机制

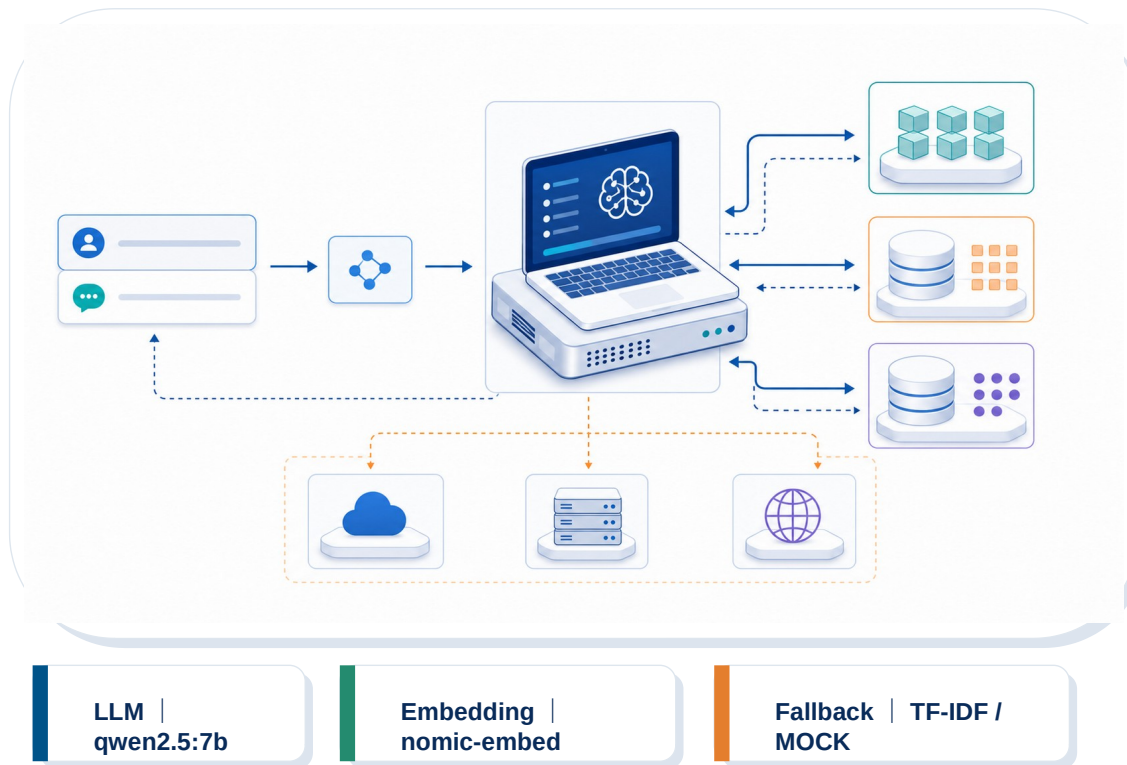
LLM： qwen2.5:7b ，用于生成面向用户的客服回答

Embedding： nomic-embed-text ，用于问题和文档片段向量化

Vector Store： 优先 ChromaDB 持久化索引，失败时退回 SimpleVectorStore

Embedding 失败时退回 TF-IDF ，保证基础检索流程仍可演示

模型生成失败时可启用 MOCK_LLM ，前端仍能展示 RAG 流程和来源



本机配置策略： qwen2.5:7b 负责生成， nomic-embed-text 负责向量化；异常时保持可演示闭环。

2 PROJECT PROCESS

前端演示页面

左侧：Ollama 状态、模型名、文档数、chunk 数、向量库类型和索引重建

中间：聊天消息、示例问题、Top-K 控制、输入框和发送状态

右侧：RAG 六步流程、检索来源、相似度、类别和内容片段

长问题、示例问题和回答区域均做换行约束，避免投屏时超出屏幕

页面不依赖 React / Vue / CDN，后端启动后即可访问



交互细节：示例问题、Top-K、状态面板和来源片段同步刷新，长文本强制换行避免超屏。

2 PROJECT PROCESS

后端 API 与代码模块

GET /api/health : 模型连接、知识库数量、 chunk 数和向量库类型

GET /api/stats : 文档数、分类分布、 chunk 数和知识库范围

POST /api/rebuild-index : 清理旧索引并重新构建本地向量库

POST /api/chat : 执行检索、 Prompt 构造、本地模型回答和来源返回

模块分层:

config、schemas、ollama_client、rag、frontend 便于维护

/api/health | 状态

/api/stats | 统计

/api/rebuild | 索引

/api/chat | 问答

main.py

retriever.py

prompt_builder.py

ollama_client.py

接口响应包含 answer、sources、metrics、steps 等字段，前端可直接渲染流程状态。

3 PROJECT RESULT

测试与验收结果

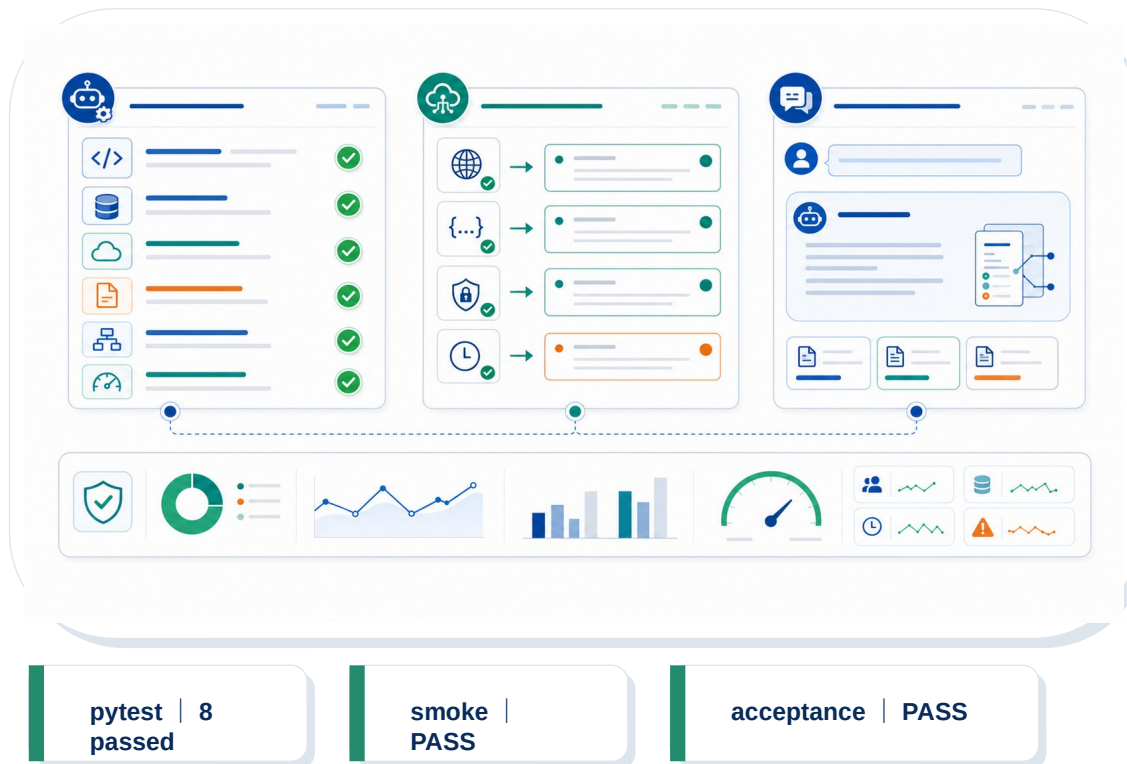
pytest：8 项单元测试通过，覆盖配置、分块、检索和接口基础行为

smoke_test：覆盖首页、健康检查、统计、重建索引和 MOCK 聊天

acceptance_check：覆盖目录、配置、文档、前端元素、降级和来源优先级

真实接口批测：资料数、RAG 流程、NXP 技术问题、越界问题和追问表达

验收重点：不是只看回答，还要看来源、流程、耗时和降级状态



验证组合：单元测试 + smoke test + acceptance check + 真实问答批测。

3 PROJECT RESULT

问题修复与迭代优化

短问法召回偏差：加入混合召回和标题覆盖评分
资料数量问题死板：改为运行上下文模式，不写硬
回答模板
重复来源：按文档聚合 sources，减少同标题
chunk 重复出现
索引重建异常：清理旧索引，修复 Chroma
readonly 场景
前端超屏：补齐长文本换行、响应式宽度和输入区
约束



迭代原则：先保证演示可用，再优化召回质量，最后处理边界问题与前端展示。

3 PROJECT RESULT

项目成果与价值

完成 FastAPI + Ollama + RAG + 原生前端完整项目
形成 17 篇知识库文档、134 个 chunks 和 Chroma
向量索引

回答带来源、相似度和处理流程，区别于普通大模型
套壳

本地化部署减少云端依赖，适合资料问答、课程演示
和边缘 AI 场景

形成 README、API 文档、设计文档、测试脚本、
报告和答辩材料

后续可扩展上传资料、增量索引、多轮会话和用户反
馈闭环

17

知识库文档

134

chunks

4

核心接口

8

单元测试

项目交付：代码仓库、知识库、向量索引、运行脚本、报告、PPT 和逐页讲解稿。

THANK YOU !

总结与展望：从课堂 Demo 到企业知识服务原型

本项目验证了本地 RAG 智能客服的技术可行性
已经覆盖知识库构建、向量索引、问答生成、来源追溯和降级演示
后续可加入知识库上传、增量索引、多轮会话和用户反馈
可进一步接入真实边缘设备资料和端侧 AI 演示
感谢各位老师指导，欢迎批评指正

资料入库

向量检索

本地生成

来源追溯

持续迭代